Neural Inverse Kinematics Trained Through a Differentiable Forward Model: A Multi-Seed Study on the UR5e

Clément Dumeril*
University of California, Berkeley (MEng)

October 2026

Abstract

A neural network can replace the inverse-kinematics (IK) solver of a robot arm, but a network trained to regress joint angles is penalised in joint space rather than where the hand lands. We train a multilayer perceptron for position IK of a UR5e at a fixed downward tool orientation and backpropagate the end-effector error through a differentiable Denavit–Hartenberg model in PyTorch. The deployed network reaches 0.191 mm mean error on 2,021 held-out targets in 0.531 ms. Over 5 seeds, adding the forward-kinematics terms to a joint-space loss lowers the test error by 11 % (95 % CI −13 to +34 %, $p = 0.26$), so an earlier single-run estimate of 21 % does not hold. On data mixing IK branches, a supervised single-output network collapses, whereas a single output trained on the forward-kinematics loss alone usually settles on one branch; extra heads trained with a relaxed winner-take-all loss, without labels, cover more branches but are not more accurate. The inherited closed-form solver returned wrong angles for every elbow-down branch (fixed here). The network is 72× faster than IKPy on the same problem but 2.3× slower than the closed form, and outside its training box it returns angles up to 1.78 m wrong without warning.

1 Introduction

Industrial arms such as the UR5e have closed-form inverse kinematics [14]: for a reachable pose the solver returns up to eight joint configurations in a fraction of a millisecond. Learned IK is therefore not needed for speed on such arms. It becomes interesting when the solver has to be differentiable, embedded in a larger learned system, or extended to arms without an analytic solution [1, 7].

The simplest learned IK regresses joint angles from Cartesian targets [8]. The loss is then expressed in joint space: an error of 10^{-3} rad on the shoulder costs as much as on the last wrist joint, although the first moves the hand by about a millimetre and the second by almost nothing. Training the inverse model through a forward model, so that the error is measured where the hand actually lands, is an old idea [15]. With automatic differentiation the forward model can be the robot's exact kinematic chain, which is what physics-informed neural networks do with differential equations [16, 24].

This paper asks two questions on a simulated UR5e pick-and-place task in Webots [21]: how much does a forward-kinematics (FK) loss term improve a position-IK network over joint regression, measured on held-out targets with several seeds; and can a network represent the multi-valued inverse without being told which solution branch each training sample belongs to. The contributions are:

*Personal project, August–September 2026. Code, data and every result file: `https://github.com/clementdumeril/ur5e-neural-kinematics`. Contact: `clementdumeril@gmail.com`.

- a paired, multi-seed ablation of the FK loss term on held-out test targets, which shows that its benefit on single-branch data is small and not significant at five seeds, contrary to an earlier single-run estimate (Section 6);
- a multi-hypothesis IK network trained without branch labels, swept over the number of heads on four-branch and eight-branch data, which separates accuracy from coverage of the solution set and shows that a single head trained on the FK loss alone can already select a branch;
- a benchmark against closed-form, damped-least-squares and IKPy solvers, a workspace error map that documents silent failure outside the training domain, and the correction of a branch-pairing error in the inherited closed-form solver.

2 Related work

Analytic and numerical IK. Kinematic chains are commonly described with Denavit–Hartenberg (DH) parameters [6, 26]. UR-series arms admit a closed-form inverse with eight branches, indexed by shoulder, wrist and elbow configuration [14]. Iterative solvers linearise the FK with the Jacobian; damped least squares keeps the step bounded near singularities [5, 22, 27]. We use both as baselines, together with the IKPy library [20].

Learned IK. D'Souza et al. [7] learn IK locally to sidestep its non-convexity; Duka [8] regress joint angles with a feed-forward network. Jordan and Rumelhart [15] train an inverse model through a learned forward model, the "distal teacher", which is the principle used here with an exact, differentiable FK in place of a learned one. Recent work computes the training loss in Cartesian space by running FK on predicted joints [12], or adds an FK-based end-effector cost to a generative model of robot configurations [19]. The term "physics-informed" comes from networks whose loss contains the residual of a governing equation [16, 24]; here the "physics" is rigid-body kinematics, with no dynamics.

Multi-valued regression. When one input has several valid outputs, a least-squares regressor predicts their mean. Mixture density networks model the conditional density [4]; normalising flows and invertible networks sample the solution set of inverse problems [2], including IK [1]; a hypernetwork can also sample joint angles one joint at a time, conditioned on the target [3]. Multiple-choice learning trains several predictors and updates only the best one per sample [11, 18]; relaxing the hard minimum avoids heads that never train [25]. Our multi-head network follows this last line.

3 Problem formulation

Kinematics. The UR5e is modelled with the standard DH convention (Table 1). The forward map $T(q) = \prod_{i=1}^{6} A_i(q_i) \in SE(3)$ gives the tool position $p(q)$ and orientation $R(q)$; d_6 includes a 0.1237 m tool offset to the gripper tip.

Task. For top-down grasping the tool orientation is fixed to $R^\star = R_{xyz}(\pi, 0, -\pi/2)$ (gripper pointing down). The learned map is therefore position IK, $f_\theta : \mathbb{R}^3 \to \mathbb{R}^6$, from a target $p^\star$ in the box $x \in [0, 0.4]$, $y \in [-0.9, -0.5]$, $z \in [0.05, 0.45]$ m above the simulated table. This is not full 6-DoF pose IK: orientation is not an input.

Table 1: DH parameters used by all solvers and by the differentiable FK. Source: `src/kinematics/ur5_pytorch_fk.py`.

i	a_i (m)	α_i	d_i (m)	θ_i offset
1	0	$\pi/2$	0.1625	0
2	0.425	0	0	$\pi/2$
3	0.3922	0	0	0
4	0	$-\pi/2$	0.1333	$-\pi/2$
5	0	$\pi/2$	0.0997	0
6	0	0	0.2233	0

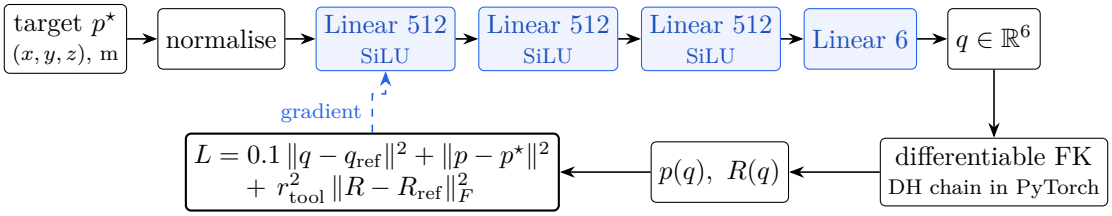


Figure 1: Single-output network and training loss. The FK is used only in the loss; inference is one forward pass. $R_{\text{ref}} = R(q_{\text{ref}})$.

Multiple solutions. For a reachable $(p^\star, R^\star)$ the closed form returns up to eight configurations, one per choice of shoulder (left/right), wrist (up/down) and elbow (up/down). A least-squares regressor trained on samples drawn from several branches converges towards their average, which in general reaches none of the targets.

4 Method

4.1 Differentiable forward kinematics

The DH chain of Table 1 is implemented as a batched PyTorch module [23]: each A_i is assembled from $\cos q_i$, $\sin q_i$ and constants, and the six transforms are chained with batched matrix products. The module agrees with the inherited NumPy FK to 2.5×10^{-16} m on 1,000 random configurations (Section 5).

4.2 Network and loss

f_θ is an MLP with three hidden layers of 512 units and SiLU activations [9] (530,438 parameters, Figure 1). SiLU was chosen because the loss gradient passes through the sines and cosines of the FK and a smooth activation keeps it continuous; it was not compared against ReLU. Input and output normalisation statistics are stored in the checkpoint. The loss is

$$L(\theta) = w_d\,\|q - q_{\text{ref}}\|^2 + w_p\left(\|p(q) - p^\star\|^2 + r_{\text{tool}}^2\,\|R(q) - R(q_{\text{ref}})\|_F^2\right), \qquad q = f_\theta(p^\star), \tag{1}$$

with $w_d = 0.1$, $w_p = 1$ and $r_{\text{tool}} = 0.05$ m, all terms averaged over the batch. Three choices matter: (i) the data term anchors the network to one IK branch; (ii) the orientation reference is $R(q_{\text{ref}})$ rather than a matrix rebuilt from Euler angles, so both sides pass through the same kinematics; without an orientation term a π rotation of q_6 flips the gripper at zero position cost; (iii) the Frobenius norm avoids the arccos of the geodesic distance, whose gradient diverges at zero error. Weighting by r_{tool}^2 converts a rotation into the displacement it causes at a 5 cm lever arm, so both terms are in m^2.

4.3 Multi-hypothesis network

For branch-mixed data a shared trunk (three layers of 512, SiLU) feeds K heads ($512 \to 256 \to 6$), giving proposals $q^{(1)}, \ldots, q^{(K)}$. With ℓ_k the FK pose error of head k (position plus r_{tool}^2 times the Frobenius rotation error, no data term), the loss is the relaxed minimum

$$L_{\text{MH}} = (1-\varepsilon)\,\ell_{k^\star} + \frac{\varepsilon}{K-1}\sum_{k \neq k^\star} \ell_k, \qquad k^\star = \arg\min_k \ell_k, \quad \varepsilon = 0.05, \tag{2}$$

so losing heads still receive gradient [25]. No branch label enters training. At inference the FK scores the K proposals with the same pose error and the best one is returned. Branch recall is the share of valid closed-form solutions that at least one head lies within 0.25 rad of, under a periodic angular distance in joint space. It measures how many branches the proposals point to, not that every proposal reaches the target: only the selected head is held to the pose error, and 0.25 rad at the shoulder moves the tool by roughly 0.2 m. With K heads and n_i valid branches for target i, recall cannot exceed $\sum_i \min(K, n_i) / \sum_i n_i$.

4.4 Data

Targets are drawn uniformly in the box. For each target the closed form is evaluated on every branch and a solution is kept only if its FK reaches the target; filtering on NaN is not enough (Section 5). The cached data set has $20,201$ reachable targets (fingerprint `b9e74b3f67cbfc10`). The deployed checkpoint was trained by `train_true_pinn.py` on a separately generated set of the same distribution, using the left–up–up branch only.

4.5 Protocol

Adam [17], learning rate 10^{-3} divided in steps to 2×10^{-5} at epochs 30, 60 and 80, batch 256, 100 epochs. All experiments in Section 6 use a fixed random split into $16,160$ training, $2,020$ validation and $2,021$ test targets; the epoch is selected on validation position error and all reported errors are test errors. Errors are the mean Euclidean distance between the pose reached by FK and the target, and the mean geodesic angle to $R(q_{\text{ref}})$. Seeds change only the initialisation and the batch order; the data and the split are the same for every run, so the reported spread does not include the variance due to data sampling.

4.6 Baselines and integration

The closed form is the inherited solver with the correction of Section 5. Damped least squares uses the geometric Jacobian with adaptive damping driven by the smallest singular value and an SO(3) logarithm for the rotation error, from one start or with eight random restarts. For IKPy [20] we build the chain from the DH table of Table 1 (its FK matches ours to 10^{-15}), impose the full tool orientation and start from the same rest configuration as damped least squares, so all solvers face the same problem on the same targets. A solution further than 1 mm or 0.1° from the requested pose counts as a failure. Timing uses 200 targets, after 200 warm-up calls on a different set. In Webots the camera reads the cube position through a homography fitted with RANSAC [10, 13] on a calibration grid, the network solves each grasp and transport target, and quintic trajectories with a closed-loop joint-position hold drive the arm.

5 Verification

Kinematic consistency. The PyTorch FK matches the NumPy FK to 2.5×10^{-16} m in position over 1,000 random configurations. The analytic geometric Jacobian, built column by column from $J_{v,i} = z_{i-1} \times (p_e - p_{i-1})$, $J_{\omega,i} = z_{i-1}$, matches a central finite-difference derivative

Table 2: FK-term ablation on single-branch data, test split, mean ± standard deviation over 5 seeds (`results/seed_study/A_*.json`).

Loss	Position (mm)	Orientation (deg)
data term only ($w_p = 0$)	0.241 ± 0.021	0.0088 ± 0.0009
data + FK terms ($w_p = 1$)	0.214 ± 0.039	0.0090 ± 0.0013
paired reduction in position	$10.8 \pm 19.0\,\%$	

of the FK to 3.2×10^{-10} (maximum absolute entry difference, 200 random configurations). At the rest pose its manipulability is 0.0287 and its condition number 16.5; at the fully extended configuration $q = 0$ the manipulability is zero, as expected.

A branch-pairing error in the closed-form solver. Round-tripping FK(IK(p)) on $2,000$ targets showed that the inherited closed form returned finite angles for every elbow-down branch, all of them wrong, up to 784 mm from the target. The solver chose θ_2 from the shoulder configuration only, while θ_3 depended on the elbow; pairing θ_2 with the elbow choice brings all eight branches to round-trip errors below 10^{-9} m. Before the fix, the FK filter of the data generator removed these solutions, so the published four-branch experiment (Part B below) only contained elbow-up branches. The fix does not change any single-branch result: the deployed model, the ablation and the benchmark use the left–up–up branch, which the error did not affect. A regression test now round-trips all eight branches.

Data and evaluation. Training, validation and test indices are disjoint; the nearest training target to a test target is 0.77 mm away at minimum and 8.4 mm at the median. Timings discard 200 warm-up calls made on separate targets, run on one thread and include the NumPy-to-tensor conversion a caller pays. A smoke test covering imports, data paths, a known target, the camera calibration and the eight-branch round trip runs in continuous integration.

6 Results

6.1 Deployed model

On the $2,021$ test targets, which it never saw, the deployed checkpoint reaches 0.191 mm mean position error (median 0.164 mm, 95th percentile 0.426 mm, maximum 1.183 mm) and 0.0087° mean orientation error. Its validation error at the selected epoch, on the set used for selection, was 0.185 mm.

6.2 Effect of the forward-kinematics terms

Table 2 and Figure 2 compare $w_p = 0$ (data term only) with $w_p = 1$ on single-branch data. The mean test position error goes from 0.241 ± 0.021 mm to 0.214 ± 0.039 mm. Paired by seed, the FK terms help in 3 of 5 seeds; the mean difference is 0.028 mm, with a 95 % confidence interval of $[-0.031, 0.086]$ mm (paired t-test, $p = 0.26$; Wilcoxon signed-rank $p = 0.31$, whose smallest attainable value with five pairs is 0.0625), and the relative reduction ranges from −16.8 to 29.6 % across seeds. Orientation error stays near one hundredth of a degree in both cases. At this sample size the FK terms do not give a reliable gain on single-branch data; detecting the observed mean difference with 80 % power at the 5 % level would take about 25 paired seeds. An earlier single run reported a 21 % reduction; it used one seed and selected and reported the epoch on the same validation set. The single-branch target is a smooth function of the position, which the data term alone already fits to a quarter of a millimetre; the run-to-run spread of the network is of the same order as the effect.

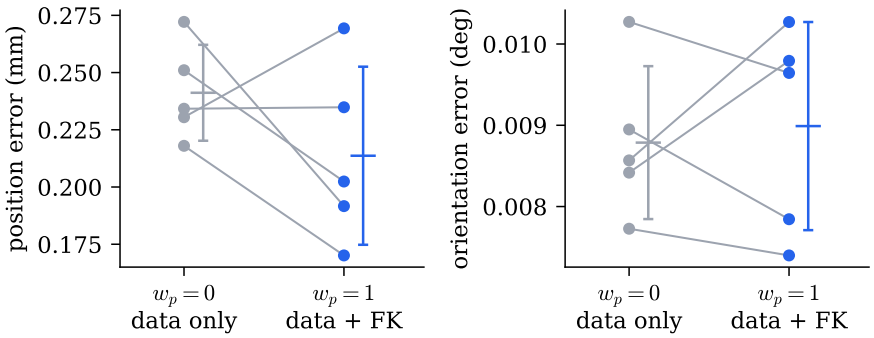


Figure 2: Test error per seed with and without the FK terms; lines join runs with the same seed, bars show mean ± one standard deviation.

Table 3: Multi-head IK on branch-mixed data, test split, mean ± standard deviation over three seeds (`results/seed_study/B_*.json, D_*.json`). Recall: share of valid branches that at least one head lies within 0.25 rad of; ceiling: highest recall attainable with K heads on the test targets.

Data	K	Position (mm)	Orientation (deg)	Recall (%)	Ceiling (%)
4 branches (B)	1	1.72 ± 2.31	1.80 ± 2.54	12.8 ± 20.9	37.1
	2	0.397 ± 0.042	0.34 ± 0.10	32.3 ± 18.4	74.2
	4	0.308 ± 0.014	0.27 ± 0.06	68.1 ± 19.2	100.0
	8	0.381 ± 0.018	0.31 ± 0.01	78.0 ± 18.7	100.0
8 branches (D)	1	3.18 ± 4.87	6.22 ± 10.24	14.2 ± 7.3	18.5
	2	0.413 ± 0.048	0.31 ± 0.02	26.6 ± 9.3	37.1
	4	0.443 ± 0.155	0.39 ± 0.20	37.8 ± 13.3	74.2
	8	0.305 ± 0.046	0.25 ± 0.05	50.6 ± 9.3	100.0

6.3 Multi-valued IK without branch labels

Table 3 and Figure 3 report the multi-head network on branch-mixed data. Part B uses the four elbow-up branches (the data set of the original experiment); Part D uses all eight branches after the solver correction, with on average 5.45 valid branches per target.

With three seeds per configuration these results are descriptive. With $K = 1$ the loss contains no data term, so nothing pulls the single output towards the average of the branches: in two of three seeds on each data set it settles on one branch and reaches 0.33–0.44 mm, and in the third it stalls between branches at 4.4 and 8.8 mm. This suggests that the collapse to the average reported in the original ablation (911 mm) comes from the supervised data term on mixed labels rather than from the single-output architecture. From $K = 2$ every run is sub-millimetre (0.26–0.62 mm) and the error does not improve systematically with more heads. Recall grows with K, from 12.8 ± 20.9 to 78.0 ± 18.7 % on four branches and from 14.2 ± 7.3 to 50.6 ± 9.3 % on eight, against ceilings of 37.1 to 100.0 % and 18.5 to 100.0 %: best-of-K needs one correct proposal, whereas covering the solution set needs one head per branch. Which branches are covered varies from seed to seed. On the four-branch set, a head that lands on an elbow-down solution counts as a miss, because those branches are absent from that data set. This is the likely reason one $K = 1$ run reaches 0.44 mm with zero recall; the weights of these runs were not stored, so it was not checked directly.

6.4 Solver benchmark

Figure 4 and Table 4 compare solve times on 200 targets. The closed form is the fastest at 0.232 ms. The single-output network takes 0.531 ms, 2.3× slower. The two generic iterative solvers behave alike from the rest configuration: damped least squares fails on 59.5 % of targets and IKPy, solving the same pose problem, on 58.0 %, both stalling in local minima. IKPy takes

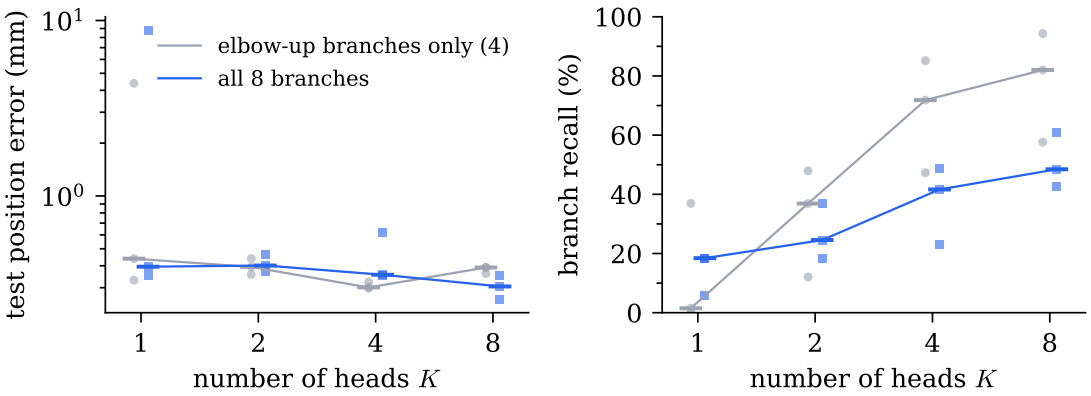
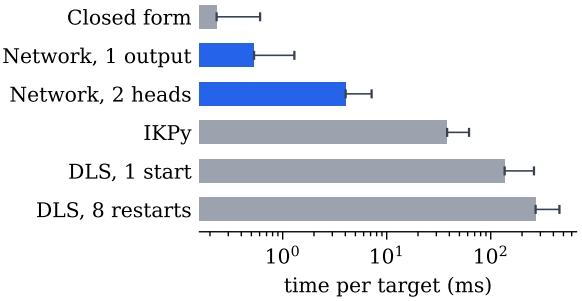


Figure 3: Multi-head sweep. Points are seeds, short bars medians. Left: test position error (log scale). Right: branch recall.



Solver	Median (ms)	p95 (ms)	Fail. (%)
Closed form	0.232	0.607	0.0
Network, 1 out.	0.531	1.296	0.0
Network, 2 heads	4.029	7.159	0.0
IKPy	38.1	61.9	58.0
DLS, 1 start	136.0	260.4	59.5
DLS, 8 restarts	270.0	457.4	0.0

Figure 4: Solve time per target (bars: median; whiskers: 95th percentile) on the same 200 pose targets, one thread, after 200 warm-up calls on other targets (`results/benchmark/benchmark.json`). Failure: more than 1 mm or 0.1° from the requested pose. Median network errors: 0.165 mm (single output) and 0.261 mm (two heads).

38.1 ms, 72× the network's time; eight restarts remove the failures of damped least squares at roughly twice its cost. The two-head network is 8× slower than the single-output one, mostly because inference must run the FK on every proposal.

6.5 Behaviour outside the training domain

Figure 5 maps the deployed network's error on a horizontal slice at $z = 0.25$ m. The median error is 0.176 mm inside the training box and 14.3 mm at reachable points outside it, 81× larger. Past the reach limit, where the closed form raises an exception, the network still returns six angles, up to 1.78 m wrong. Along a line at $y = -0.70$ m, $z = 0.25$ m the error of the returned pose is 0.2 mm at $x = 0.40$, 2.0 mm at 0.50, then 68, 389 and 1460 mm at 0.55, 0.60 and 0.70 m, where no solution exists (`results/verification/continuity_log.txt`).

Continuity. Along a 400-sample circle inside the workspace, the network and the closed form give identical joint-space steps (median 0.0084 rad) and no branch change. With the tool pointing down the preferred branch is valid everywhere on the path, so the network has no continuity advantage here.

6.6 Closed-loop simulation

In Webots (Figure 6) the arm locates the cube with its camera and the network solves every move to the cube and to the tray; only the camera reading pose, outside the training box, uses the closed form. A campaign that drives the arm to the same target with both solvers and

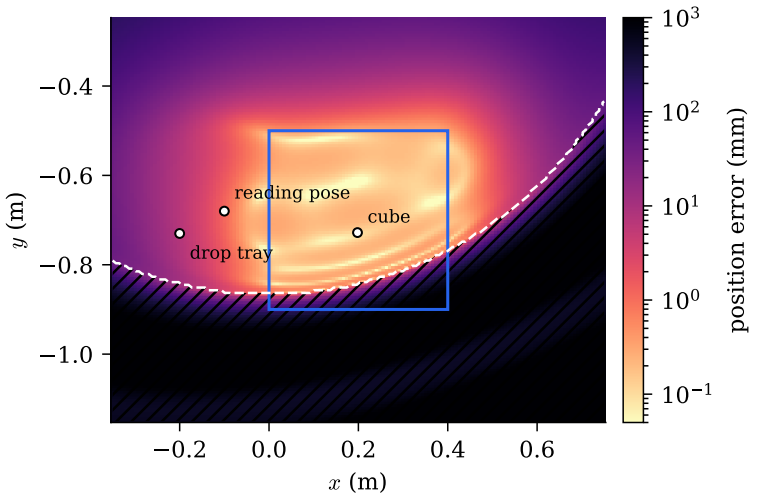


Figure 5: Position error of the deployed network at $z = 0.25\,\mathrm{m}$ (log colour scale). Blue: training box. Dashed: reach limit of the closed form; hatched: no solution exists.

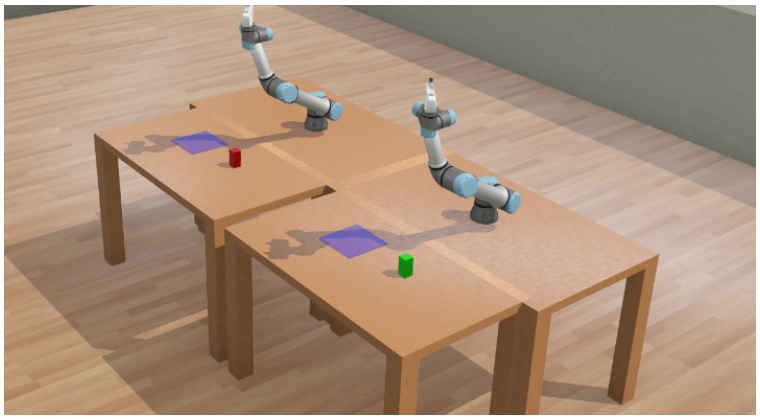


Figure 6: Webots comparison scene: one arm driven by the network (red cube), one by the closed form (green cube).

compares the poses reached in the physics engine measured 0.30 mm at the grasp point and 7 to 11 mm above the tray. The tray, at $x = -0.20\,\mathrm{m}$, lies outside the training box, yet the network drives the arm there. These figures were obtained with an earlier checkpoint trained without the orientation term and have not been repeated with the deployed one. A homography fitted on a calibration grid maps pixels to the table with a 1.1 mm median residual. A colour-threshold detector located the cube within 8 mm, against 49 mm for a VGG16 regressor, which was dropped; since a file-path error that silently disabled the camera was fixed, the camera loop has not been re-run in simulation.

7 Discussion and limitations

What the results support. A position-IK network trained through a differentiable FK reaches about 0.2 mm on held-out targets inside its training box. A loss computed on the reached pose, rather than on joint labels, lets the network cope with a multi-valued inverse: one head picks a branch on its own most of the time, and several heads trained without labels cover a growing share of the solution set.

What they do not support.

- *A gain from the FK terms on single-branch data.* With five seeds the paired difference is not significant; more seeds or a larger effect would be needed to claim one.
- *Pose IK.* The network maps positions to joints at one fixed tool orientation. Full 6-DoF IK would take the orientation as input and needs new data.
- *Speed.* The network is 2.3× slower than the closed form. Its speed advantage holds only against generic iterative solvers.
- *Safety.* Outside the training box the error grows by orders of magnitude and the network gives no signal. A reachability and domain check must sit in front of it; the present pick-and-place loop does not have one and sends the network to the tray, outside its training box.
- *Closed-loop accuracy of the deployed model.* The only closed-loop measurement predates the deployed checkpoint, and the camera loop has not been re-run since the path fix.
- *Multi-head trade-off.* On this task, fixing the branch in the data generator is simpler and more accurate than learning it; the multi-head model becomes useful when the branch cannot be chosen in advance, for instance with several tool orientations or obstacle constraints. Which branches the heads cover varies between seeds.
- *Physical validity of branches.* Branches are counted when the FK reaches the target; joint limits and collisions with the table are not checked, so some elbow-down solutions may not be executable.
- *Scope of measurement.* Results come from kinematic evaluation and one simulator; the closed-loop figures predate the deployed checkpoint, and the workspace map is a single horizontal slice.
- *Naming.* The loss contains no dynamics or differential operator; "physics-informed" here means kinematics-informed.

8 Conclusion and future work

On a single IK branch, backpropagating the end-effector error through an exact differentiable FK did not give a reliable gain over joint regression once seeds and a held-out test set were taken into account. On mixed data, only a pose loss trains without labels; a single head avoided the collapse to the average in four of six runs, and a relaxed winner-take-all multi-head network covered more of the solution set, at no gain in accuracy. The network remains slower than the closed form and unsafe outside its domain. Natural next steps are an orientation input for pose IK, an explicit reachability or confidence output, joint-limit and collision checks on the recovered branches, and repeating the closed-loop measurement with the deployed checkpoint.

Reproducibility statement

Code, checkpoints, cached-data fingerprints and all result files are in the repository. Hardware: Intel Core i7-6700HQ (4 cores), 32 GB RAM, no GPU. Software: Python 3.11.9, PyTorch 2.8.0 (CPU), NumPy 2.2.6, SciPy 1.15.2, Matplotlib 3.10.6, IKPy 4.0.0, Webots R2025a, MiKTeX pdfTeX 4.23.

Command	Time
`python experiments/verify_paper.py`	1 min
`python experiments/seed_study.py` (Parts A–C)	3 h
`SEED_PART=D python experiments/seed_study.py`	2.5 h
`python experiments/benchmark_solveurs.py`	6 min
`python experiments/mesure_continuite.py`	1 min
`python paper/make_figures.py && python paper/make_numbers.py`	4 min
`cd paper && latexmk -pdf main.tex`	1 min

Every number in this paper is either copied by hand from the file cited next to it or generated into `paper/numbers.tex` by `paper/make_numbers.py`.

References

[1] Barrett Ames, Jeremy Morgan, and George Konidaris. IKFlow: Generating diverse inverse kinematics solutions. *IEEE Robotics and Automation Letters*, 7(3):7177–7184, 2022. doi: 10.1109/LRA.2022.3181374.

[2] Lynton Ardizzone, Jakob Kruse, Sebastian Wirkert, Daniel Rahner, Eric W. Pellegrini, Ralf S. Klessen, Lena Maier-Hein, Carsten Rother, and Ullrich Köthe. Analyzing inverse problems with invertible neural networks. In *International Conference on Learning Representations (ICLR)*, 2019. URL `https://arxiv.org/abs/1808.04730`.

[3] Raphael Bensadoun, Shir Gur, Nitsan Blau, and Lior Wolf. Neural inverse kinematic. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 1787–1797, 2022. URL `https://proceedings.mlr.press/v162/bensadoun22a.html`.

[4] Christopher M. Bishop. Mixture density networks. Technical Report NCRG/94/004, Aston University, Birmingham, UK, 1994. URL `https://publications.aston.ac.uk/id/eprint/373/`.

[5] Samuel R. Buss. Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods. Unpublished survey, Department of Mathematics, University of California, San Diego, 2009. URL `https://mathweb.ucsd.edu/~sbuss/ResearchWeb/ikmethods/iksurvey.pdf`. Version dated October 7, 2009.

[6] J. Denavit and R. S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955. doi: 10.1115/1.4011045.

[7] A. D'Souza, S. Vijayakumar, and S. Schaal. Learning inverse kinematics. In *Proceedings of the 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 1, pages 298–303, Maui, HI, USA, 2001. IEEE. doi: 10.1109/IROS.2001.973374.

[8] Adrian-Vasile Duka. Neural network based inverse kinematics solution for trajectory tracking of a robotic arm. *Procedia Technology*, 12:20–27, 2014. doi: 10.1016/j.protcy.2013.12.451.

[9] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018. doi: 10.1016/j.neunet.2017.12.012.

[10] Martin A. Fischler and Robert C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981. doi: 10.1145/358669.358692.

[11] Abner Guzman-Rivera, Dhruv Batra, and Pushmeet Kohli. Multiple choice learning: Learning to produce multiple structured outputs. In *Advances in Neural Information Processing Systems 25 (NIPS 2012)*, pages 1799–1807, 2012. URL https://proceedings.neurips.cc/paper_files/paper/2012/hash/cfbce4c1d7c425baf21d6b6f2babe6be-Abstract.html.

[12] Jan-Gerrit Habekost, Erik Strahl, Philipp Allgeuer, Matthias Kerzel, and Stefan Wermter. CycleIK: Neuro-inspired inverse kinematics. In *Artificial Neural Networks and Machine Learning – ICANN 2023*, Lecture Notes in Computer Science, pages 457–470. Springer Nature Switzerland, 2023. doi: 10.1007/978-3-031-44207-0_38.

[13] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, UK, 2nd edition, 2004. doi: 10.1017/CBO9780511811685.

[14] Kelsey P. Hawkins. Analytic inverse kinematics for the Universal Robots UR-5/UR-10 arms. Technical report, Georgia Institute of Technology, 2013. URL http://hdl.handle.net/1853/50782.

[15] Michael I. Jordan and David E. Rumelhart. Forward models: Supervised learning with a distal teacher. *Cognitive Science*, 16(3):307–354, 1992. doi: 10.1207/s15516709cog1603_1.

[16] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021. doi: 10.1038/s42254-021-00314-5.

[17] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015. URL https://arxiv.org/abs/1412.6980.

[18] Stefan Lee, Senthil Purushwalkam Shiva Prakash, Michael Cogswell, Viresh Ranjan, David Crandall, and Dhruv Batra. Stochastic multiple choice learning for training diverse deep ensembles. In *Advances in Neural Information Processing Systems 29 (NIPS 2016)*, pages 2119–2127, 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/hash/20d135f0f28185b84a4cf7aa51f29500-Abstract.html.

[19] Teguh Santoso Lembono, Emmanuel Pignat, Julius Jankowski, and Sylvain Calinon. Learning constrained distributions of robot configurations with generative adversarial network. *IEEE Robotics and Automation Letters*, 6(2):4233–4240, 2021. doi: 10.1109/LRA.2021.3068671.

[20] Pierre Manceron. IKPy. Software, https://github.com/Phylliade/ikpy, 2024. Concept DOI; version 4.0.0 was used in this work.

[21] Olivier Michel. Cyberbotics Ltd. Webots™: Professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1), 2004. doi: 10.5772/5618.

[22] Yoshihiko Nakamura and Hideo Hanafusa. Inverse kinematic solutions with singularity robustness for robot manipulator control. *Journal of Dynamic Systems, Measurement, and Control*, 108(3):163–171, 1986. doi: 10.1115/1.3143764.

[23] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In

Advances in Neural Information Processing Systems 32 (NeurIPS 2019), pages 8026–8037, 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[24] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

[25] Christian Rupprecht, Iro Laina, Robert DiPietro, Maximilian Baust, Federico Tombari, Nassir Navab, and Gregory D. Hager. Learning in an uncertain world: Representing ambiguity through multiple hypotheses. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 3611–3620. IEEE, 2017. doi: 10.1109/ICCV.2017.388.

[26] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Advanced Textbooks in Control and Signal Processing. Springer, London, 2009. doi: 10.1007/978-1-84628-642-1.

[27] Charles Wampler. Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1):93–101, 1986. doi: 10.1109/TSMC.1986.289285.